

Computer Hardware

Threads and Synchronisation in C++

Ksawery Moźdzynski



WORK IN PROGRESS

Warning: this presentation is still under construction.
It has not been run and tested yet.



Agenda For Today



1. Threads Several threads, one program,
many cores



2. The race Why `counter++` loses updates



3. Mutexes One thread at a time



4. Atomic operations Three steps as
one, and what it costs

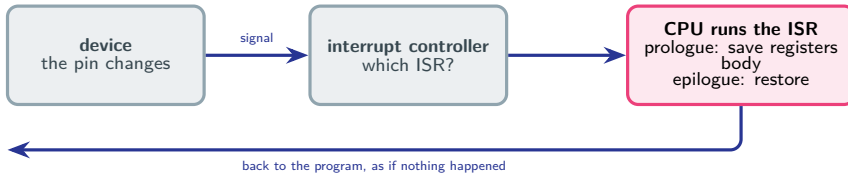


5. The bank A bigger program, and a new
problem



6. Back on the ESP32 The same mutex
in FreeRTOS

From Monday: interrupts

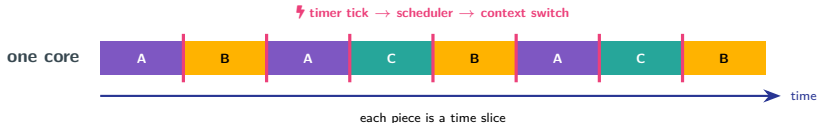


Rules for an ISR

1. keep it short
2. never wait
3. share data carefully (volatile)

The CPU saves what the ISR changes, so the program never notices the interruption.

From Monday: the scheduler



Context switch

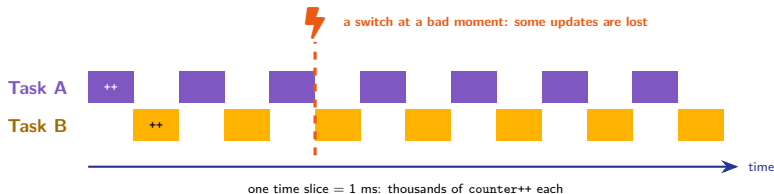
Save the registers of task A on **A's own stack**, then restore the registers of task B. B continues where it stopped.

Choosing the next task

Each task is **running**, **ready** or **blocked**. The ready task with the **highest priority** runs.

FreeRTOS does this on the ESP32-C3 all the time: even `loop()` is a task.

From Monday: a race condition



⚠ **Two cores**
Two ISRs run presses++ at the **same moment**. One press is lost.

⚠ **One core and a scheduler**
A switch falls between two tasks that both run counter++. Some additions are lost.

A **race condition**: the result depends on **when** the tasks touch the shared data.

The question

counter++ is **one line**.

How can a switch fall *in the middle* of it?

Today: the answer, and the fix. But first, a compiler on your laptop.

Task 0: C++ on your laptop

Task

Compile and run `hello.cpp`, the program from “Before Tuesday” (also on [GitHub](#)).



```
#include <iostream>
#include <thread>

int main() {
    std::thread t([] {
        std::cout << "Hello from a thread\n";
    });
    t.join();
}
```

g++ or clang++ (Linux, macOS, MinGW)



```
g++ -std=c++20 -pthread hello.cpp -o hello
./hello
```

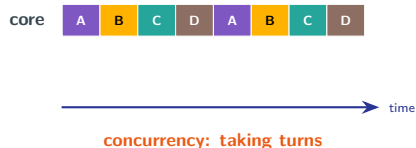
MSVC (Developer Command Prompt): `cl /std:c++20 /EHsc hello.cpp`, then `hello.exe`

New in `hello.cpp`: `std::thread` starts a thread, `join()` waits for it, `[] { ... }` is a function without a name (a *lambda*). All three in a moment.

From today we write C++ on our own laptops: **many cores, real threads**.
Does it not compile? **Raise your hand now**, not in an hour.

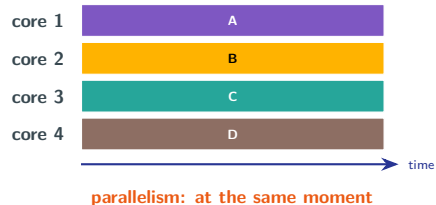
One core takes turns, many cores work at once

ESP32-C3: 1 core



Concurrency: tasks make progress by **taking turns**. This is what we saw on Monday.

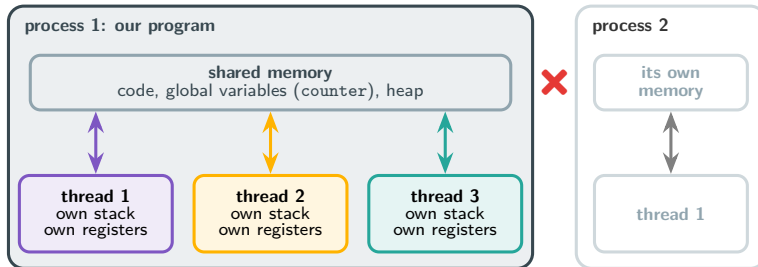
laptop: 4 cores



Parallelism: tasks run **at the same moment** on different cores. On Monday we only gave it a name. Today we use it.

A laptop does both: the cores work in parallel, and when there are more tasks than cores, they take turns.

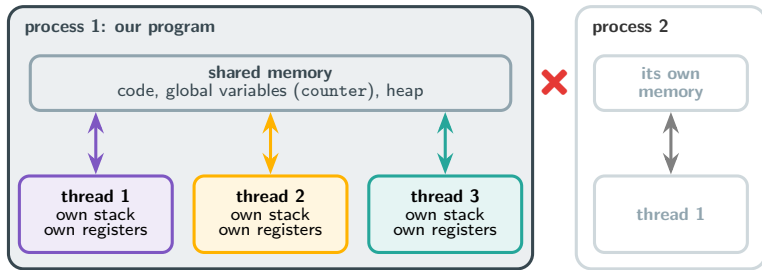
Processes and threads: a process



Process

A running program with **its own memory**: its code, its global variables, its heap. Other processes cannot see it. Your browser and your compiler are two processes.

Processes and threads: a thread



Thread

A path of execution **inside** a process. The threads of one process **share** its memory, but each has its own stack and registers.

A thread is the laptop's version of Monday's **task**: its own context, shared global variables.

Task 1: how many cores?



```
#include <iostream>
#include <thread>

int main() {
    std::cout << std::thread::hardware_concurrency()
               << "\n";
}
```

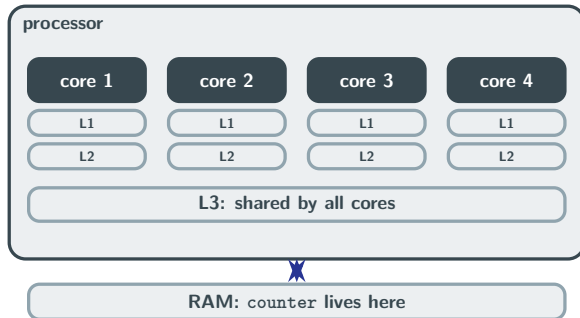
New: `std::thread::hardware_concurrency()` returns the number of threads the hardware can run at the same moment.

Task

Compile and run the program (also on [GitHub](#)).

1. How many threads can your laptop run **at the same moment**?
2. Check how many **cores** your processor has: Task Manager (Windows), `sysctl -n hw.physicalcpu` (macOS), `lscpu` (Linux). Is it the same number?

Solution: how many cores?



Question 1

The number of **logical** cores: often 4 to 16.

Question 2

Often **twice** the number of cores: with *hyper-threading* (Intel, AMD) one core runs 2 threads. Apple M chips have no hyper-threading: the numbers are the same.

More threads than this number? They **take turns**: Monday's scheduler, now inside the operating system.

Creating a thread

```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
void hello(int id) {  
    std::cout << "Hello from thread " << id << "\n";  
}  
  
int main() {  
    std::thread t(hello, 1);  
    std::cout << "Hello from main\n";  
    t.join();  
}
```

The whole program

main starts a thread that runs `hello(1)`, prints its own line, and waits for the thread.

Two new things

`std::thread` and `join`. Let us look at them **one by one**.

Creating a thread: `std::thread`

```
● ● ●
#include <iostream>
#include <thread>

void hello(int id) {
    std::cout << "Hello from thread " << id << "\n";
}

int main() {
    std::thread t(hello, 1);
    std::cout << "Hello from main\n";
    t.join();
}
```

Start a thread

A new thread starts **at once** and runs `hello(1)`: a function and its argument. `main` does not wait: it goes on to the next line at the same time.

Like `xTaskCreate` on Monday: a function and a parameter. The operating system schedules it.

Creating a thread: join

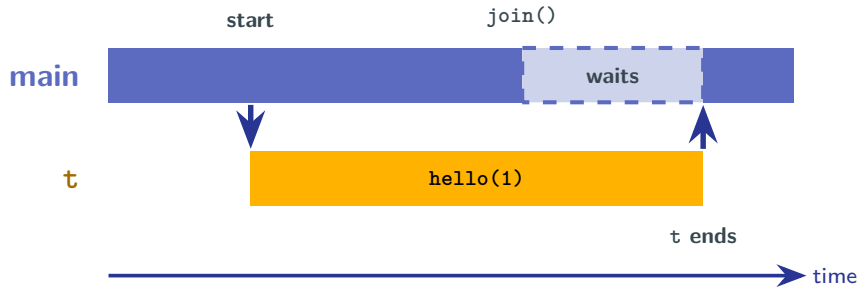
```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
void hello(int id) {  
    std::cout << "Hello from thread " << id << "\n";  
}  
  
int main() {  
    std::thread t(hello, 1);  
    std::cout << "Hello from main\n";  
    t.join();  
}
```

Wait for the thread

main stops here until t ends.

Every thread must be joined before its `std::thread` object is destroyed, here: before the end of main.

Creating a thread: start and join



```
std::thread t(hello, 1)
t starts. From now on main and t run at the same time.
```

```
t.join()
main waits until t ends, then goes on.
```

Many threads: `std::vector`

```
● ● ●  
  
#include <iostream>  
#include <thread>  
#include <vector>  
  
void hello(int id) {  
    std::cout << "Hello from thread " << id << "\n";  
}  
  
int main() {  
    std::vector<std::thread> threads;  
    for (int i = 0; i < 4; i++)  
        threads.emplace_back(hello, i);  
    for (auto &t : threads)  
        t.join();  
}
```

Four threads in a vector

`emplace_back(hello, i)` creates a new thread at the end of the vector. It starts at once and runs `hello(i)`.

The argument is **copied**: each thread gets its own copy of `i`.

The second loop joins them all.

Many threads: `std::ref`

```
● ● ●
#include <iostream>
#include <thread>
#include <functional>

void add(int i, int &total) {
    total += i;
}

int main() {
    int total = 0;
    std::thread t(add, 5, std::ref(total));
    t.join();
    std::cout << total << "\n";    // 5
}
```

Pass a reference

Arguments are copied, so without `std::ref` the thread would change its own copy of `total`.

`std::ref(total)` passes a **reference**: the thread changes the variable of `main`.

In `hello.cpp` you saw `[] { ... }` instead of a function name: a *lambda*, a function without a name, written in place.

Task 2: four threads say hello

Task

Write a program that starts **4 threads**. Thread `i` prints `Hello from thread i`. Template: [GitHub](#).

1. Run it **5 times**. Is the order of the lines always the same?
2. Do you ever see two messages **mixed** in one line?
3. Remove the loop with `join()`. What happens?

If you finish early: pass `total` to all 4 threads with `std::ref`. Thread `i` adds `i` to it. Print `total` after the joins. Do you always get 6?

Solution: four threads say hello



```
Hello from thread 1
Hello from thread 0
Hello from thread 3
Hello from thread 2
```



```
Hello from thread Hello from thread 2
0
```

Question 1

No. The operating system decides when each thread runs.

Question 2

Sometimes yes: << prints in **pieces**, and another thread can print between them.

Question 3

It crashes: `terminate` called without an active exception. Every thread must be joined.

If you finished early

Usually 6, but **not guaranteed**: see the next task.

With threads the order of events is **not fixed**. A program must be correct in **every** order.

Task 3: Monday's counter in C++

race.cpp

```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
int counter = 0;  
const int N = 1'000'000;  
  
void work() {  
    for (int i = 0; i < N; i++)  
        counter++;  
}  
  
int main() {  
    std::thread a(work), b(work);  
    a.join(); b.join();  
    std::cout << "Goal: " << 2 * N << "\n";  
    std::cout << "Got:  " << counter << "\n";  
}
```

Task

Download `race.cpp` from [GitHub](#). Compile it with `-O0`: no optimisation, so every `counter++` really reads and writes memory (MSVC: `/Od`).

1. Run it **5 times**. Write down the results.
2. Start only **one** thread and print `N` as the goal. Is the result correct now?

If you finish early: compile with `-O2` instead. What happens to the result, and to the time?

Solution: Monday's counter in C++

Two threads, 5 runs (a test machine)

Run	Got
1	1 322 869
2	1 484 050
3	1 334 267
4	2 000 000
5	1 863 852

Question 1

Usually **too small**, and different every time. Sometimes even correct: the first thread ended before the second one started.

Question 2

One thread: **always** correct.

If you finished early

With -O2 the compiler adds N at once, without the loop. The result is (almost always) correct, but the bug is still there.

On Monday one core took turns. Today two cores work **at the same moment**. Same bug. Now: **why?**

counter++ in the processor

C++

● ● ●
counter++;



RISC-V (simplified)

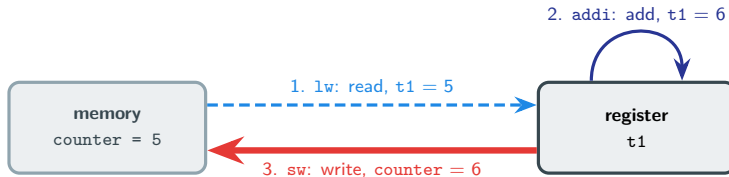
● ● ●
`la t0, counter` *# t0 = the address of counter*
`lw t1, 0(t0)` *# t1 = counter*
`addi t1, t1, 1` *# t1 = t1 + 1*
`sw t1, 0(t0)` *# counter = t1*

You saw this on Monday

In Lecture 6 (“Inside the CPU: 3. the body”) the ISR ran presses++ with exactly these four instructions. counter++ is the same. What a real compiler makes of it: in a moment, in Compiler Explorer.

la only finds the variable. The work is **three** instructions: lw, addi, sw.

counter++: read, add, write



Why three steps?

The processor cannot add to a value **in memory**. It reads the value into a register, adds there, and writes it back. x86 and ARM do the same inside.

counter++: between the instructions

One core

The scheduler can **switch** to the other thread after `lw` or after `addi`. The other thread runs, and only later the first one finishes its `sw`. This is what happened on Monday.

Two cores

No switch is needed: the other thread runs its own `lw`, `addi`, `sw` **at the same moment**, on its own core.

One line of C++, but **three** steps for the processor. Things can happen **between** them.

One update lost, step by step

Thread A

```
▶ la    t0, counter
lw      t1, 0(t0)
addi    t1, t1, 1
sw      t1, 0(t0)
```

registers of A

t0 = 0x2000

t1 = ?

memory

0x2000

counter = 5

Thread B

```
▶ la    t0, counter
lw      t1, 0(t0)
addi    t1, t1, 1
sw      t1, 0(t0)
```

registers of B

t0 = 0x2000

t1 = ?

Step 1: both threads run la

Each thread has its **own** registers. Both now have $t0 = 0x2000$, the address of counter. counter is 5.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000 t1 = 5

memory

0x2000

counter = 5

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000 t1 = ?

Step 2: A runs lw

A reads counter into its own t1: t1 = 5.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000

t1 = 5

memory

0x2000

counter = 5

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000

t1 = 5

Step 3: B runs lw

B reads counter into its own t1: also 5, because A has not written anything yet.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000

t1 = 6

memory

0x2000

counter = 5

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000

t1 = 5

Step 4: A runs addi

A's t1 = 6. counter in memory is still 5.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000

t1 = 6

memory

0x2000

counter = 5

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000

t1 = 6

Step 5: B runs addi
B's t1 = 6 too.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000 t1 = 6

memory

0x2000

counter = 6

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000 t1 = 6

Step 6: A runs sw

A writes its t1: counter = 6.

One update lost, step by step

Thread A

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of A

t0 = 0x2000

t1 = 6

memory

0x2000

counter = 6

Thread B

```
la    t0, counter
lw    t1, 0(t0)
addi  t1, t1, 1
sw    t1, 0(t0)
```

registers of B

t0 = 0x2000

t1 = 6

Step 7: B runs sw

B writes its t1: counter = 6 **again**. Two additions, but counter grew by **one**.

One update lost: two additions, but +1



B wrote over the result of A

Each thread worked on **its own copy** of counter, in its own register t1. B read the value **before** A wrote its result, so A's addition is lost.

This is the answer to Monday's puzzle: two presses, but presses grew by one.

One update lost: why so many?

A thread stopped between lw and sw

On one core, a thread may be switched out right after its lw. Then the other thread runs a whole time slice and adds **thousands** of times. When the first one comes back, it writes its old value + 1 over all of them.

That is why Task 3 lost not one or two, but **hundreds of thousands** of additions.

Simulator 8: race step by step

Simulator task

Topic **Race step by step**: threads A and B each run `lw, addi, sw` once. `counter = 5`. You choose which thread runs the next instruction.

1. Find an order that gives **7**. Then one that gives **6**.
2. Can you get **5**? Why?
3. How many of all possible orders give the correct result?

 ksawerym.github.io/ComputerHardwareSimulators/#cc-race

Solution: Simulator 8

Question 1

7: one thread does all three instructions first (A A A B B B). 6: both read before the other one writes (A B A B A B, ...).

Question 2

No. Each thread adds 1 to a value it has read, and that value is at least 5.

Question 3

There are 20 orders. Only 2 are correct: A A A B B B and B B B A A A.

18 of 20 orders are wrong. That is why the error appears **so often**.

Compiler Explorer: counter++

Task

Open godbolt.org. Paste the code below. Compiler: **RISC-V (32-bits) gcc**, option **-O1**.

```
int counter;  
void inc() { counter++; }
```

1. Find `lw`, `addi` and `sw`. Which registers does the compiler use?
2. Our slides had `la t0, counter`. What is there instead?
3. Change the compiler to **x86-64 gcc**. How many instructions now? Is it safe now?

Solution: Compiler Explorer

RISC-V gcc -O1



```
inc:
    lui  a4,%hi(counter)
    lw   a5,%lo(counter)(a4)
    addi a5,a5,1
    sw   a5,%lo(counter)(a4)
    ret
```

x86-64 gcc -O1



```
inc:
    add DWORD PTR counter[rip], 1
    ret
```

Question 1

The same three steps. The compiler chose a4 for the address and a5 for the value: any free register will do.

Question 2

1a is two instructions: lui (the upper part of the address) and addi (the lower part). Here the lower part %lo(counter) went straight into lw and sw.

Question 3

One instruction, but still **not safe**: inside the core it is still read, add, write, and another core can come in between.

A name for it: race condition

Race condition: the result depends on the **timing** of the threads.

You know this name from Monday

There the timing came from the scheduler. Today it also comes from two cores working at the same moment. The bug is the same.

A name for it: data race

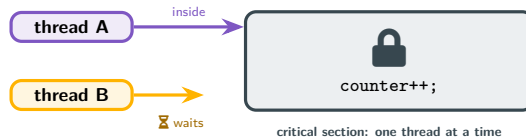
Data race: two threads use the same variable at the same time, and at least one of them **writes**.

 In C++ a data race is undefined behaviour

The C++ standard says nothing about what such a program does. Anything may happen: lost updates, wrong values, or a program that works today and fails tomorrow.

A name for it: critical section

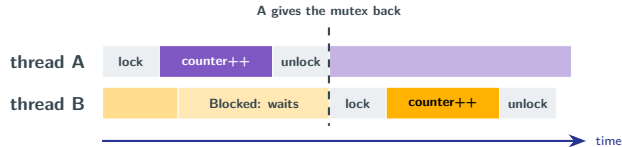
Critical section: a piece of code that must not run in two threads at the same time. Here: `counter++`.



The fix: make sure that only **one thread at a time** is inside the critical section.

A mutex: lock and unlock

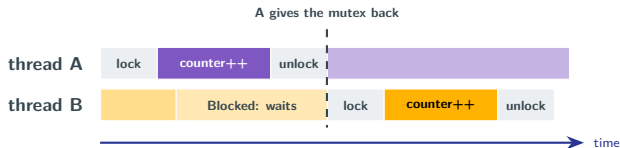
A **mutex** (*mutual exclusion*) is a lock. `lock()`: wait until it is free, then take it. `unlock()`: give it back.



Only one owner

While A holds the mutex, B cannot take it. B can enter only after A's `unlock()`.

A mutex: waiting costs nothing



B is Blocked

A thread that waits for a mutex is **Blocked**, like a task in `vTaskDelay` on Monday. The scheduler does not give it any time: it uses no CPU time.

A gives the mutex back

Then B becomes **Ready**, takes the mutex, and runs its `counter++`.

Simulator 9: a mutex

Simulator task

Topic **Mutex**: the same two threads, but now each runs `lock, lw, addi, sw, unlock`. `counter = 5`.

1. Try to find an order that gives **6**.
2. Thread A holds the mutex. Click thread B. What happens? In which state is B?
3. How many instructions can come **between** A's `lw` and A's `sw` now?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-mutex](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-mutex)

Solution: Simulator 9

Question 1

Impossible: **every** order gives 7.

Question 2

B stops at lock. It is **Blocked** until A calls `unlock`.

Question 3

None of thread B. The three steps of A now act like **one** step.

The mutex removes all the wrong orders. Now let us write it in C++.

A mutex in C++

```
● ● ●  
  
#include <iostream>  
#include <thread>  
#include <mutex>  
int counter = 0;  
const int N = 1'000'000;  
std::mutex m;  
  
void work() {  
    for (int i = 0; i < N; i++) {  
        std::lock_guard<std::mutex> lock(m);  
        counter++;  
    }  
}  
  
int main() {  
    std::thread a(work), b(work);  
    a.join(); b.join();  
    std::cout << "Got: " << counter << "\n";  
}
```

The whole program

Task 3, with counter++ protected by a mutex.

Three new things

std::mutex, std::lock_guard, and the **end of the block**. Let us look at them **one by one**.

A mutex in C++: `std::mutex`

```
● ● ●  
  
#include <iostream>  
#include <thread>  
#include <mutex>  
  
int counter = 0;  
const int N = 1'000'000;  
std::mutex m;  
  
void work() {  
    for (int i = 0; i < N; i++) {  
        std::lock_guard<std::mutex> lock(m);  
        counter++;  
    }  
}
```

The mutex

One mutex, a **global** variable: both threads must see the **same** one. It starts unlocked. (`#include <mutex>`)

One mutex protects **one** piece of shared data, here counter. Every thread that uses the data must lock **the same** mutex.

A mutex in C++: `std::lock_guard`



```
#include <iostream>
#include <thread>
#include <mutex>

int counter = 0;
const int N = 1'000'000;
std::mutex m;

void work() {
    for (int i = 0; i < N; i++) {
        std::lock_guard<std::mutex> lock(m);
        counter++;
    }
}
```

Lock

A local variable `lock`. When it is created, it calls `m.lock()`: wait until `m` is free, then take it.

`<std::mutex>`: the type of the mutex it locks.

A mutex in C++: the end of the block

```
● ● ●  
  
#include <iostream>  
#include <thread>  
#include <mutex>  
  
int counter = 0;  
const int N = 1'000'000;  
std::mutex m;  
  
void work() {  
    for (int i = 0; i < N; i++) {  
        std::lock_guard<std::mutex> lock(m);  
        counter++;  
    }  
}
```

Unlock

Here the block { ... } ends, and the local variable `lock` is destroyed. Its destructor calls `m.unlock()`.

So in every iteration: `lock`, `counter++`, `unlock`.

Why not `m.lock()` and `m.unlock()`?

```

● ● ●
const int LIMIT = 1'500'000;

void work() {
    for (int i = 0; i < N; i++) {
        m.lock();
        if (counter >= LIMIT)
            return;    // m stays locked!
        counter++;
        m.unlock();
    }
}

```

⚠ It is easy to forget `unlock()`

Here after the early return. The mutex stays locked, and every other thread waits **forever**.

With `std::lock_guard`

The block ends on **every** path, also after return, so the mutex is always unlocked.

Measuring the time



```
#include <chrono>

// in main():
auto start = std::chrono::steady_clock::now();
// start and join the threads
auto end = std::chrono::steady_clock::now();
std::chrono::duration<double, std::milli> ms
    = end - start;
std::cout << ms.count() << " ms\n";
```

New in this code

`steady_clock::now()`: the time now.

`end - start`: how long it took.

`duration<double, std::milli>`: that time in milliseconds.

`ms.count()`: the number itself.



Not `clock()` from Lecture 4

It adds up the CPU time of **all** threads.

Task 4: fix it with a mutex, and measure

Task

Start from Task 3 (-00), or the template from [GitHub](#). The template already measures the time.

1. Add a mutex. Is the result correct now?
2. Measure the time **without** and **with** the mutex.
3. How many times slower is the correct version?

If you finish early: put the `lock_guard` **before** the loop, so that the whole loop is locked. Correct? How fast? What do the threads do now?

Solution: fix it with a mutex

Two threads (a test machine)

Version	Got	Time
no protection	1 334 267	6 ms
mutex	2 000 000	200 ms

Question 1

Yes: exactly 2 000 000, every time.

Questions 2 and 3

About **30 times** slower. Every `counter++` now needs `lock` and `unlock`, and the threads wait for each other all the time.

If you finished early

Correct and fast, but the threads run **one after the other**: no parallelism at all.

Correct, but slow. Can we make the three steps **one** step instead?

Atomic operations

Atomic: the read, the add and the write happen as **one** step. No other thread can come in between.

No lock needed

The hardware itself makes the three steps one. Nobody waits for a mutex.

An atomic works for **one** variable and simple operations. For more than that, we need a mutex.

Atomic operations: `std::atomic`

```
● ● ●
#include <iostream>
#include <thread>
#include <atomic>

std::atomic<int> counter{0};
const int N = 1'000'000;

void work() {
    for (int i = 0; i < N; i++)
        counter++;           // one step
}

int main() {
    std::thread a(work), b(work);
    a.join(); b.join();
    std::cout << "Got: " << counter << "\n";
}
```

An atomic int

counter is now a `std::atomic<int>`, starting at 0. `counter++` on it is **one** atomic step.

No mutex, no `lock_guard`: the rest of the code is Task 3. (`#include <atomic>`)

Atomic operations: inside the CPU

RISC-V, counter++ on a `std::atomic<int>` (gcc -O1)

```

● ● ●
lui      a5,%hi(counter)
li       a4,1           # a4 = 1
addi     a5,a5,%lo(counter) # a5 = address
amoadd.w .aqrl zero,a4,0(a5) # counter += a4
  
```

lui + addi

The la from before: a5 = the address of counter.

amoadd.w

Atomic memory operation: read, add a4, write, as **one** instruction. zero: we do not need the old value.

.aqrl

Rules about the order of memory accesses. Not needed today.

For other operations

lr.w / sc.w: read, change, and write only *if nobody wrote in between*; if someone did, try again.

Task 5: atomic, and a better idea

Task

Two more versions of the counter. Measure the time of each.
Template: [GitHub](#).

1. **Atomic:** `std::atomic<int>` counter, without the mutex.
2. **Local sum:** each thread counts in its **own local variable**, and adds it to the shared counter **once**, at the end (with the mutex).
3. Fill in the table. Which versions are correct? Which is the fastest correct one?

Version	Correct?	Time
no protection		
mutex		
atomic		
local sum		

If you finish early: in Compiler Explorer, make counter a `std::atomic<int>` and choose **x86-64 gcc, -O1**. Which instruction do you see now?

Solution: atomic, and a better idea

Two threads (a test machine)

Version	Correct?	Time
no protection	✗	6 ms
mutex	✓	200 ms
atomic	✓	30 ms
local sum	✓	2.5 ms

If you finished early

lock add DWORD PTR counter[rip], 1. The lock prefix makes the x86 add atomic: no other core can come in between.

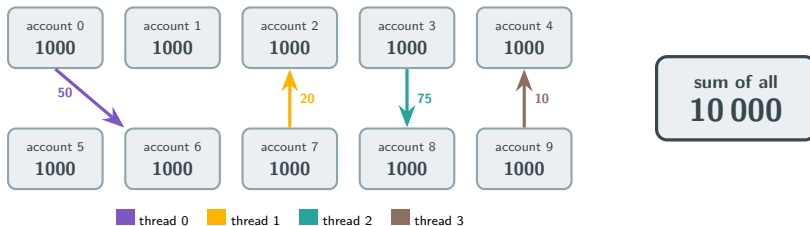
Local sum



```
void work() {  
    int local = 0;  
    for (int i = 0; i < N; i++)  
        local++; // nothing shared  
    std::lock_guard<std::mutex> lock(m);  
    counter += local; // once per thread  
}
```

The best synchronisation is the one you do not need: **share less**.

The bank



A transfer

Pick two accounts and an amount. Subtract it from one account, add it to the other. Several threads do this at the same time.

The **sum** of all balances must always be **10 000**. Easy to check: if the sum is wrong, the program is wrong.

Negative balances are allowed: we keep the bank simple.

Bank, step 1: the code



```
const int ACCOUNTS = 10, THREADS = 4;
const int TRANSFERS = 1'000'000 / THREADS;
int balance[ACCOUNTS];

void transfers(int id) {
    std::mt19937 rng(id);
    std::uniform_int_distribution<int>
        acc(0, ACCOUNTS - 1), amount(1, 100);
    for (int i = 0; i < TRANSFERS; i++) {
        int from = acc(rng), to = acc(rng);
        if (from == to) continue;
        int x = amount(rng);
        balance[from] -= x;
        balance[to] += x;
        if (i % 50'000 == 0)
            std::cout << id << ": " << i << "\n";
    }
}
```

New in this code

`std::mt19937 rng(id)`: a generator of random numbers. Its start value is `id`, so each thread gets different numbers.

`uniform_int_distribution<int> acc(0, 9)`: a random int from 0 to 9.

`acc(rng)`: draw one. (`#include <random>`)

`bank.cpp: main()`

Not shown here. Sets every balance to 1000, starts `THREADS` threads, joins them, and prints the **sum** and the **time**.

Bank, step 1: no synchronisation

Task

Download `bank.cpp` from [GitHub](#). Compile it with `-O0`.

1. Run it several times. What is the sum?
2. Set `THREADS = 1`. And now?

Solution, step 1: money disappears

4 threads (a test machine)



Sum: 37637 time: 100 ms

Sum: -117645 time: 115 ms

1 thread



Sum: 10000 time: 110 ms

Question 1

The sum is wrong, and different every time. Money **disappears** or **appears**.

Question 2

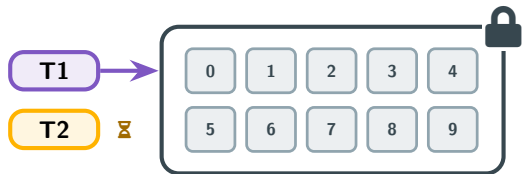
With one thread the sum is always 10 000.

⚠ Why?

`balance[from] -= x` is the same **read, change, write** as `counter++`.

Lock granularity: one big lock, or many small ones?

One big lock

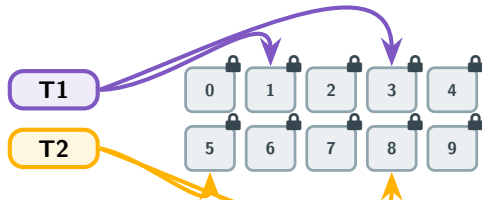


one transfer at a time

Simple and safe

But threads wait for each other, even when they use different data.

A lock for each piece of data



different accounts: at the same time

More work in parallel

Threads that use different data do not wait. But it is harder to get right.

We try both: step 2 with one lock for the whole bank, step 3 with a lock for each account.

Bank, step 2: one mutex for the whole bank



```
std::mutex bank;
```

```
// in transfers(), around the transfer:
```

```
std::lock_guard<std::mutex> lock(bank);
```

```
balance[from] -= x;
```

```
balance[to]   += x;
```

Task

Add one `std::mutex` for the whole bank.

1. Is the sum correct now?
2. Measure the time with `THREADS = 1, 2 and 4`.
The total number of transfers stays the same.
Does it get faster with more threads?

Solution, step 2: correct, but a queue

One mutex (a test machine)

Threads	Sum	Time
1	10 000	150 ms
2	10 000	210 ms
4	10 000	200 ms

Question 1

Yes: always 10 000.

Question 2

No. More threads are **not** faster, even a bit slower: only one transfer at a time, and the others wait at the lock.

A transfer from 1 to 2 and a transfer from 5 to 8 do not need to wait for each other. Idea: **one mutex per account**.

Bank, step 3: one mutex per account



```
std::mutex m[ACCOUNTS];
```

```
// in transfers(), around the transfer:
```

```
std::lock_guard<std::mutex> a(m[from]);
```

```
std::lock_guard<std::mutex> b(m[to]);
```

```
balance[from] -= x;
```

```
balance[to]   += x;
```

Task

Replace the bank mutex with a mutex for **each account**. A transfer locks its two accounts.

1. Run it several times. Does it always finish?
2. Watch the progress messages. What does the program do? Look at the CPU usage (Task Manager, Activity Monitor, top).

Solution, step 3: the program stops

Question 1

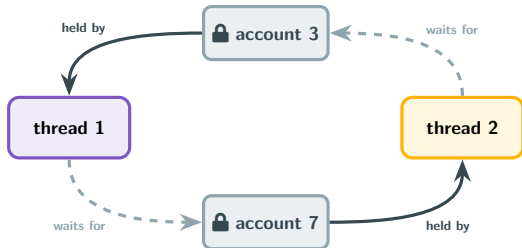
No. It **hangs**: the messages stop, and the program never ends.

Question 2

It uses **no** CPU time. The threads are not working: they are **Blocked**, each one waiting for a mutex.

Every thread waits. But for what?

Deadlock



What happened

Thread 1 transfers from account 3 to 7, thread 2 from 7 to 3. Each locked its from account, and now waits for the other one.

Deadlock: each thread waits for a lock that another thread holds. Nobody can continue.

The question

How can a thread lock **two** accounts without this happening? **Tomorrow.**

A mutex in FreeRTOS



```
SemaphoreHandle_t m;
```

```
void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        xSemaphoreTake(m, portMAX_DELAY);    // lock
        counter++;
        xSemaphoreGive(m);                    // unlock
    }
    *(volatile bool *)param = true;          // done
    vTaskDelete(NULL);                        // end this task
}
```

```
void setup() {
    // the LCD as on Monday
    m = xSemaphoreCreateMutex();
    // the two tasks as on Monday
}
```

The whole program

Monday's Task 5, with counter++ protected by a mutex.

Three new things

xSemaphoreCreateMutex, xSemaphoreTake and xSemaphoreGive. Let us look at them **one by one**.

A mutex in FreeRTOS: xSemaphoreCreateMutex

```
● ● ●  
SemaphoreHandle_t m;  
  
void incTask(void *param) {  
    for (int i = 0; i < N; i++) {  
        xSemaphoreTake(m, portMAX_DELAY);    // lock  
        counter++;  
        xSemaphoreGive(m);                    // unlock  
    }  
    *(volatile bool *)param = true;          // done  
    vTaskDelete(NULL);                        // end this task  
}  
  
void setup() {  
    // the LCD as on Monday  
    m = xSemaphoreCreateMutex();  
    // the two tasks as on Monday  
}
```

Create the mutex

It creates a mutex and returns a *handle* to it, of the type `SemaphoreHandle_t`. We keep it in the global `m`.

Create it **before** the tasks that use it.

In FreeRTOS a mutex is a kind of *semaphore*: more about them tomorrow.

A mutex in FreeRTOS: xSemaphoreTake

```
SemaphoreHandle_t m;

void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        xSemaphoreTake(m, portMAX_DELAY); // lock
        counter++;
        xSemaphoreGive(m);                // unlock
    }
    *(volatile bool *)param = true; // done
    vTaskDelete(NULL);              // end this task
}

void setup() {
    // the LCD as on Monday
    m = xSemaphoreCreateMutex();
    // the two tasks as on Monday
}
```

Lock

Like `m.lock()`: take the mutex, or wait until it is free.

`portMAX_DELAY`: wait as long as needed. While it waits, the task is **Blocked**.

A mutex in FreeRTOS: xSemaphoreGive

```
● ● ●
SemaphoreHandle_t m;

void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        xSemaphoreTake(m, portMAX_DELAY);    // lock
        counter++;
        xSemaphoreGive(m);                    // unlock
    }
    *(volatile bool *)param = true;          // done
    vTaskDelete(NULL);                       // end this task
}

void setup() {
    // the LCD as on Monday
    m = xSemaphoreCreateMutex();
    // the two tasks as on Monday
}
```

Unlock

Like `m.unlock()`: give the mutex back.
If the other task waits for it, it becomes Ready.

There is no `lock_guard` in C: do not forget this line.

Task 6: fix Monday's Task 5

Task

Open the template from [GitHub](#): Monday's Task 5, already solved. Protect `counter++` with a FreeRTOS mutex.

1. Create the mutex in `setup()`, and lock and unlock it around `counter++`.
2. Run it several times. What does the LCD show now?
3. Is it faster or slower than on Monday? Why?

If you finish early: use the idea of the local sum from Task 5. How many times does each task take the mutex now?

Solution: fix Monday's Task 5

Question 2

Got: 2000000, every time.

Question 3

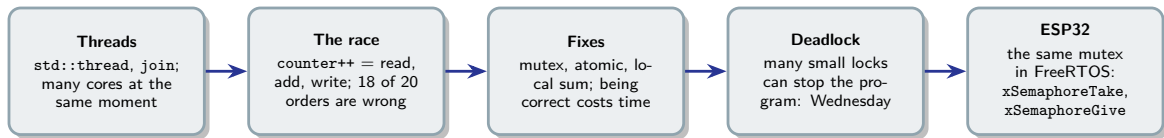
Slower: two million times `xSemaphoreTake` and `xSemaphoreGive`, and the tasks wait for each other. The same as Task 4 today.

If you finished early

Each task counts in a local variable and takes the mutex **once**, at the end. Correct and fast.

The same idea everywhere: FreeRTOS, C++, every operating system.

What we saw today



Monday's puzzle is solved: counter++ is not one step, and a lock makes it one.

Check yourself

1. What is the difference between a process and a thread? What do the threads of one process share?
2. Why does `counter++` lose updates? Use the words *read*, *add* and *write*.
3. Why does `std::lock_guard` unlock the mutex even after an early return?
4. Why is the mutex version correct but slow? What did the local-sum version do better?
5. What is a deadlock? Draw the situation from step 3 of the bank.

Keep your `bank.cpp`: on Wednesday we repair step 3.